

RAG con SQL Database de Fabric

Roberto Carrancio



Roberto Carrancio



- DBA as a Service - @Encora
- 15 años trabajando con SQL Server en proyectos de todos los tamaños y sectores.
- Microsoft Data Platform MVP (SQL Server y Azure SQL)
- Consultoría y formación a medida.
- Autor del libro "**SQL SERVER: La NO guía práctica de optimización**"
- Creador y autor del blog www.soydba.es
- Co-Administrador del grupo de Telegram SQL Server Español



<https://www.linkedin.com/in/robertocarrancio>



<https://www.youtube.com/@soydba>



<https://www.soydba.es>



Agenda

- ¿Que es Fabric SQL DB, RAG y vector / embedding?
- ¿Por qué RAG?
- Funciones de IA de SQL Server
- ¿Qué es DiskANN ?
- DEMO RAG y Observabilidad

Fundamentos tecnológicos



Fabric SQL DB

La base de datos SaaS de Microsoft en la plataforma Fabric. Todo el poder de SQL Server, gestionado y optimizado para la era de la IA.



RAG

Retrieval Augmented Generation. Inyectamos nuestro contexto corporativo (datos privados) al LLM antes de que genere su respuesta.



Embeddings

Vectores numéricos que capturan el significado semántico profundo de textos, permitiendo comparar conceptos por su "distancia".

¿Por qué RAG frente a Búsqueda Tradicional?

La Búsqueda Tradicional (Léxica)

Históricamente, los motores de búsqueda dependían de la **coincidencia exacta de palabras clave** (keywords).

Ignora por completo la intención real del usuario.

Sufre de rigidez extrema ante sinónimos o errores tipográficos.

Devuelve listas de enlaces o documentos, obligando al usuario a leer y extraer la respuesta manualmente.

La Revolución RAG (Semántica)

RAG busca por significado y proximidad conceptual, no por cadenas de texto exactas.

Entiende el contexto, los sinónimos y la jerga empresarial.

Recupera fragmentos de información altamente relevantes (Chunks).

El LLM razona sobre esos fragmentos para devolver una respuesta natural y fundamentada, sin alucinaciones.

Tipo de datos Vector en SQL

- Diseñado para almacenar vectores y optimizado para búsquedas de similitud.
- Se almacena en un formato binario optimizado, pero se representa como un array JSON.
- Son tipos de datos muy grandes, **mejor almacenar en otra tabla.**
- Usan un tipo base float32. Se puede elegir float16 (half-precision).
- El límite es de 1998 dimensiones en base 32 y 3996 en base 16.
- Disponible en:
 - SQL Server 2025
 - Azure SQL Database
 - Azure SQL Managed Instance
 - Fabric SQL Database

Generación de embeddings

`sp_invoke_external_endpoint`

- Permite llamar a un endpoint externo para generar una respuesta mediante un servicio de IA alojado fuera de la base de datos.
- Envía el texto, recibe el JSON resultante.
- Podemos integrar cualquier servicio HTTPS REST.

`AI_GENERATE_EMBEDDINGS`

- Función escalar nativa que usa `sp_invoke_external_endpoint` para llamar a un modelo externo creado en la base de datos.
- Los modelos, creados con `CREATE EXTERNAL MODEL` deben de ser de tipo `EMBEDDING`

CREATE EXTERNAL MODEL

- Crea un objeto de modelo externo de IA.
- Puede ser:
 - Azure OpenAI
 - OpenAI
 - Ollama
 - ONNX Runtime
- Siempre HTTPS

```
CREATE EXTERNAL MODEL external_model_object_name
[ AUTHORIZATION owner_name ]
WITH
(
  LOCATION = '<prefix>://<path>[:<port>]'
  , API_FORMAT = '<OpenAI, Azure OpenAI, etc>'
  , MODEL_TYPE = EMBEDDINGS
  , MODEL = 'text-embedding-model-name'
  [ , CREDENTIAL = <credential_name> ]
  [ , PARAMETERS = '{"valid":"JSON"}' ]
  [ , LOCAL_RUNTIME_PATH = 'path to the ONNX runtime files' ]
);
```

TIPO_DE_MODELO

Tipo de modelo al que se accede desde la ubicación del punto de conexión de inferencia del modelo de IA.

Los valores aceptados son:

- EMBEDDINGS

MODELO

Modelo específico hospedado por el proveedor de IA. Por ejemplo, text-embedding-ada-002, text-embedding-3-large o o3-mini.

AI_GENERATE_EMBEDDINGS

- Función escalar para crear embedding
- Necesita un modelo creado con:
CREATE EXTERNAL MODEL
- El modelo solo puede ser de embeddings

```
SELECT id,  
       AI_GENERATE_EMBEDDINGS(t.TextoLargo USE MODEL MyAzureOpenAIModel)  
FROM Tabla ;
```

```
SELECT AI_GENERATE_EMBEDDINGS('Aqui pongo mi texto largo' USE MODEL MyAzureOpenAIModel)
```

sp_invoke_external_endpoint

Procedimiento almacenado para llamadas REST

```
EXECUTE @returnValue = sp_invoke_external_rest_endpoint
[ @url = ] N'url'
[ , [ @payload = ] N'request_payload' ]
[ , [ @headers = ] N'http_headers_as_json_array' ]
[ , [ @method = ] 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE' | 'HEAD' ]
[ , [ @timeout = ] seconds ]
[ , [ @credential = ] credential ]
[ , @response OUTPUT ]
[ , [ @retry_count = ] # of retries if there are errors ]
```

sp_invoke_external_endpoint

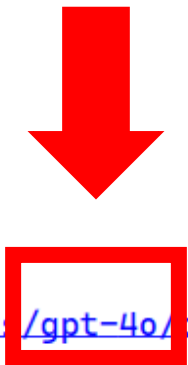
```
create master key encryption by password = N'NoEsMiContras3ñ@';

create database scoped credential [https://testaisoydba.openai.azure.com] -- Nombre de la credencial (Endpoint Azure OpenAI)
with identity = 'HTTPEndpointHeaders', secret = '{"api-key": "*****"}'; -- API KEY

declare @payload nvarchar(max) =
json_object(
    'messages': json_array(
        json_object(
            'role': 'user',
            'content': 'Hola GPT'
        )
    )
);

declare @response nvarchar(max)
exec sp_invoke_external_rest_endpoint
    @url = 'https://testaisoydba.openai.azure.com/openai/deployments/gpt-4o/chat/completions?api-version=2025-01-01-preview',
    @method = 'POST',
    @credential = [https://testaisoydba.openai.azure.com], -- Nombre de la credencial (Endpoint Azure OpenAI)
    @payload = @payload,
    @response = @response output

select @response
```



```
"result": {
  "choices": [
    {
      "content_filter_results": {
        "hate": {
          "filtered": false,
          "severity": "safe"
        },
        "protected_material_code": {
          "filtered": false,
          "detected": false
        },
        "protected_material_text": {
          "filtered": false,
          "detected": false
        },
        "self_harm": {
          "filtered": false,
          "severity": "safe"
        },
        "sexual": {
          "filtered": false,
          "severity": "safe"
        },
        "violence": {
          "filtered": false,
          "severity": "safe"
        }
      },
      "finish_reason": "stop",
      "index": 0,
      "logprobs": null,
      "message": {
        "annotations": [],
        "content": "\u00A1Hola! \u00BFEn qu\u00E9 puedo ayudarte hoy? \uD83D\uDE0A",
        "refusal": null,
        "role": "assistant"
      }
    }
  ]
}
```



"content": "\u00A1Hola! \u00BFEn qu\u00E9 puedo ayudarte hoy? \uD83D\uDE0A",

AI_GENERATE_CHUNKS

- Función con valores de tabla que crea "fragmentos" o fragmentos de texto basados en un tipo, tamaño y expresión de origen.
- Solo admite tipos de chunks FIXED
- Se puede especificar tamaño y overlap (en %) de chunk
- Se puede activar una columna chunk_set_id

Vector_Distance

- Función escalar que calcula la distancia entre vectores.
- **NO USA LOS INDICES VECTORIALES**
- Puede usar métricas de distancia:
 - Euclidean
 - Dot
 - Cosine
- Sintaxis:

```
VECTOR_DISTANCE ( distance_metric , vector1 , vector2 )
```

VECTOR_SEARCH

- Función de tabla que busca el Top N vectores más cercanos.
- **USA LOS INDICES VECTORIALES.**
- **Solo FILTRO POSTERIOR.** La búsqueda de vectores se produce antes de aplicar cualquier predicado.
- Puede usar métricas de distancia Euclidean, Dot y Cosine
- Sintaxis:

```
VECTOR_SEARCH(  
  TABLE = object [ AS source_table_alias ]  
  , COLUMN = vector_column  
  , SIMILAR_TO = query_vector  
  , METRIC = { 'cosine' | 'dot' | 'euclidean' }  
  , TOP_N = k  
) [ AS result_table_alias ]
```

DiskANN: Búsqueda Vectorial

DiskANN (basado en el algoritmo de grafo

Vamana de Microsoft Research) revoluciona la indexación vectorial superando los límites de la RAM.



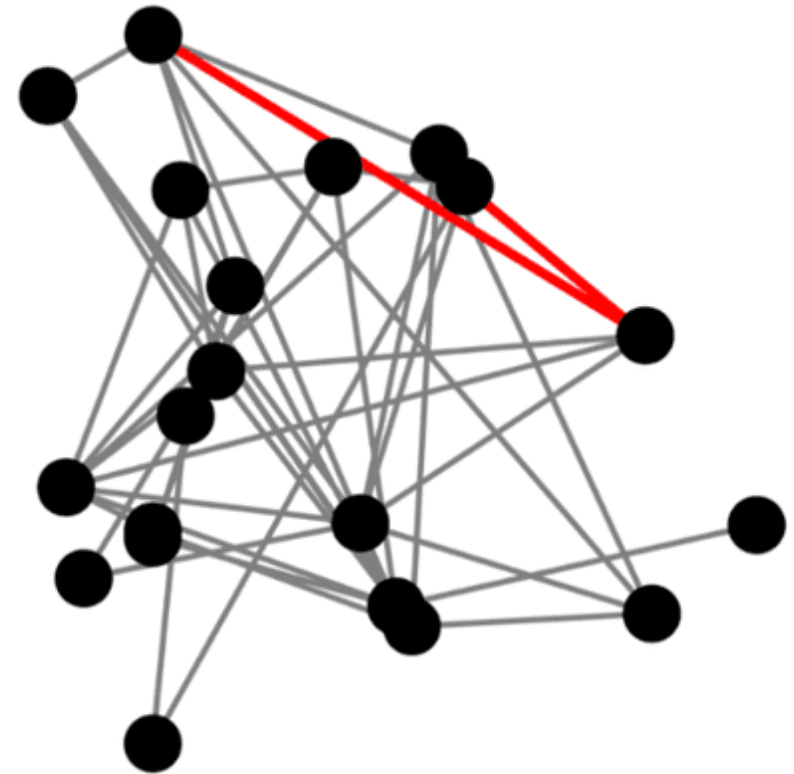
Optimizado para SSD: Mantiene la conectividad del grafo en disco sin penalizar severamente la latencia.



Navegación de Grafo: Ruta eficiente desde un nodo de entrada hasta los vecinos más cercanos.



Regla de Oro: Solo aporta beneficio de rendimiento real cuando superas los 50.000 registros vectoriales.



El Filtro en DiskANN

DiskANN filtra **antes** de los filtros tradicionales en SQL (Post-filtering).

- 1 Fase 1 (Vectorial):** El motor de SQL utiliza el índice DiskANN para encontrar rápidamente el Top K de vecinos más cercanos basándose únicamente en el Embedding.
 - 2 Fase 2 (Relacional):** Sobre ese Top K resultante, se aplican a posteriori los filtros tradicionales definidos en el WHERE (ej. Categoría = 'snack').
- ! El Riesgo:** Si ninguno de los elementos del Top K vectorial cumple la condición tradicional del WHERE, la consulta devolverá un resultado vacío, aunque existan registros válidos más abajo en el ranking general.
 - 💡 Arquitectura:** Es crítico diseñar la partición de datos y afinar el valor de "K" considerando este comportamiento.

Preparando el RAG

1. Desplegar modelos en Foundry
2. En SQL
 1. Habilitar external rest endpoint
 2. Crear clave maestra
 3. Crear credencial con la API Key de Foundry
 4. Generar embeddings
 5. Crear SP búsqueda
 1. Embedding de la pregunta
 2. Búsqueda vectorial
 3. Llamada API a LLM para generar respuesta
 4. LOG detallado
3. Crear cuadro de mando

A · D · N  **Fabric**

DEMO

Buenas prácticas

- No expongas API Keys en SPs, siempre usa una credencial.
- No almacenes vectores en la misma tabla que los datos.
- Generación de embeddings siempre asíncrono.
- Lo que no se registra no se puede controlar.
- Cuidado con datos confidenciales y regulaciones.

A · D · N  Fabric

¡Gracias!

A · D · N  **Fabric**

¿Preguntas?

Recursos

- SQL Vector Data Type
<https://learn.microsoft.com/en-us/sql/t-sql/data-types/vector-data-type>
- AI Generate Chunks
<https://learn.microsoft.com/en-us/sql/t-sql/functions/ai-generate-chunks-transact-sql>
- Create External Model
<https://learn.microsoft.com/en-us/sql/t-sql/statements/create-external-model-transact-sql>
- AI Generate Embeddings
<https://learn.microsoft.com/en-us/sql/t-sql/functions/ai-generate-embeddings-transact-sql>
- Vector Search
<https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-search-transact-sql>
- Vector Distance
<https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-distance-transact-sql>
- Vector Indexes
<https://learn.microsoft.com/en-us/sql/sql-server/ai/vectors>
<https://learn.microsoft.com/en-us/sql/t-sql/statements/create-vector-index-transact-sql>
- FAQ IA Microsoft
[Intelligent Applications and AI Frequently Asked Questions \(FAQ\) - SQL Server | Microsoft Learn](#)
- Guía **Getting started with AI in SQL Server 2025 on Windows**
<https://github.com/AzureSQLDB/SQLin5/blob/main/newAIinSQL2025/getting-started-with-ai.md>